

Livrable 1 - Construction et alimentation d'une infrastructure de gestion de données

Livrable attendu : Une étude de 1 page décrivant schématiquement l'infrastructure conceptualisée et le code source permettant de construire l'infrastructure.

1. Table des matières

- 1. Table des matières
- 2. Introduction
 - 2.1. Contexte
 - 2.2. Problématique
 - 2.3. Objectifs de l'infrastructure
- 3. Analyse des besoins et contraintes
 - 3.1. Collecte et centralisation des données
 - 3.2. Préparation des données
 - 3.3. Traçabilité des traitements
 - 3.4. Déploiement reproductible
- 4. Principes d'architecture retenus
 - 4.1. Couche de stockage
 - 4.2. Couche de métadonnées
 - 4.3. Couche de traitement
 - 4.4. Couche Data Science
 - 4.5. Couche applicative
- 5. Mise en oeuvre du Data Lake et des processus ETL
 - 5.1. Collecte et intégration des données
 - 5.2. Organisation du Data Lake
 - 5.3. Construction des données d'entraînement
 - 5.4. Gestion des métadonnées
 - 5.4.1. PostgreSQL
 - 5.4.2. MongoDB
- 6. Préparation à la montée en charge
- 7. Infrastructure as Code et reproductibilité
- 8. Industrialisation et qualité logicielle
 - 8.1. Validation du code
 - 8.2. Construction et publication de l'application
- 9. Schéma synthétique
- 10. Évolutivité de la plateforme
 - 10.1. Calcul distribué
 - 10.2. MLOps
 - 10.3. Valorisation des résultats
- 11. Conclusion

2. Introduction

2.1. Contexte

Le projet a pour objectif de développer une solution capable de transcrire automatiquement un enregistrement de guitare en une représentation musicale, telle qu'une partition ou un fichier MIDI, en utilisant des techniques de traitement du signal et de Machine Learning.

La conception d'un système de transcription nécessite la mise en place d'une infrastructure capable de collecter, stocker, transformer et tracer les données utilisées pour l'entraînement des modèles, tout en garantissant la reproductibilité des expérimentations et le déploiement du modèle sélectionné.

L'architecture globale du projet est conçue afin de couvrir l'ensemble des besoins, depuis la collecte des données jusqu'à l'exploitation du modèle au travers d'une application web.

La [note de cadrage](#) et le [cahier des charges](#) présentent plus en détail le projet.

2.2. Problématique

Le développement du projet présente plusieurs défis techniques.

Les données d'apprentissage proviennent de sources hétérogènes et sont constituées de fichiers audio, d'annotations musicales et de métadonnées associées. Leur préparation nécessite plusieurs étapes de traitement successives (prétraitement audio, extraction de caractéristiques fréquentielles, alignement des annotations et construction des jeux de données d'entraînement), dont les résultats doivent rester reproductibles.

Par ailleurs, les expérimentations conduisent à entraîner plusieurs modèles de Machine Learning utilisant des architectures différentes. Il est indispensable de conserver l'ensemble des paramètres, métriques et artefacts produits afin de comparer objectivement les performances obtenues et de sélectionner le modèle le plus pertinent.

Enfin, le modèle sélectionné doit pouvoir être intégré dans une application permettant aux utilisateurs de soumettre un fichier audio et d'obtenir automatiquement les résultats de la transcription. Cette dernière étape implique de disposer d'une infrastructure de déploiement garantissant la portabilité de l'application, la reproductibilité de son environnement d'exécution et l'automatisation des opérations de validation et de mise en production.

Problématique : Comment concevoir une infrastructure de données reproductible, évolutive et industrialisable permettant de gérer l'ensemble du cycle de vie d'un projet de Machine Learning, depuis la collecte des données jusqu'au déploiement d'un modèle de transcription musicale ?

2.3. Objectifs de l'infrastructure

Afin de répondre à la problématique, l'infrastructure doit valider les objectifs ci-après :

- centraliser le stockage des données brutes, des jeux de données construits, des modèles et des artefacts de traitement,

- assurer la traçabilité des jeux de données et des pipelines de prétraitement afin de garantir la reproductibilité des expérimentations,
- faciliter l'entraînement, la comparaison et le versionnement des modèles de Machine Learning grâce à une plateforme dédiée au suivi des expérimentations,
- standardiser l'environnement de développement afin de garantir des conditions d'exécution identiques entre les différents environnements (développement, intégration continue et production),
- automatiser les opérations de validation, de construction et de déploiement au moyen d'une chaîne d'intégration et de déploiement continu (CI/CD),
- proposer une architecture modulaire permettant de faire évoluer la plateforme sans remettre en cause son organisation globale.

L'ensemble de ces objectifs a conduit à concevoir une architecture reposant sur des composants spécialisés, orchestrés par **Docker Compose** et entièrement décrits sous forme de code (Infrastructure as Code), afin de garantir la reproductibilité, la maintenabilité et l'évolutivité de la solution.

3. Analyse des besoins et contraintes

L'analyse des besoins a conduit à identifier quatre fonctions principales.

3.1. Collecte et centralisation des données

Les données proviennent de sources hétérogènes possédant des formats différents (WAV, JAMS et XML). L'infrastructure doit permettre leur téléchargement automatique puis leur intégration dans un espace de stockage unique afin de constituer un **Data Lake**.

L'[inventaire des sources de donnees](#) et le [descriptif des données](#) présentent les différentes sources collectées.

3.2. Préparation des données

Les fichiers audio doivent subir plusieurs traitements avant d'être exploitables :

- nettoyage du signal,
- normalisation,
- extraction des caractéristiques musicales (features),
- alignement avec les annotations,
- génération d'échantillons destinés à l'apprentissage.

Ces traitements sont réalisés au travers de pipelines ETL développées en Python.

3.3. Traçabilité des traitements

Chaque transformation appliquée aux données doit être historisée afin de garantir la reproductibilité des expérimentations.

Les métadonnées enregistrées comprennent notamment :

- les pipelines exécutées,
- les paramètres utilisés,
- les fichiers produits,

- les échantillons générés,
- les jeux de données construits.

Cette traçabilité est assurée par **MongoDB**, tandis que les expérimentations de Machine Learning sont suivies par **MLflow**.

3.4. Déploiement reproductible

L'ensemble des services nécessaires au projet doit pouvoir être reconstruit automatiquement sur un nouvel environnement.

L'infrastructure est donc entièrement décrite sous forme de code à l'aide de **Docker Compose**, tandis que le déploiement applicatif est automatisé par une chaîne **CI/CD GitHub Actions**.

4. Principes d'architecture retenus

L'architecture a été conçue selon une approche en couches, chaque composant remplissant une responsabilité clairement identifiée.

4.1. Couche de stockage

Le stockage repose sur un **Data Lake MinIO** organisé en plusieurs buckets :

Bucket	Rôle
raw	stockage des données sources
processed	stockage des données nettoyées, features et échantillons
mlflow	stockage des artefacts des expérimentations
output	réservé aux productions futures de l'API

Cette organisation permet de séparer les différents états de transformation des données tout en conservant les fichiers originaux.

4.2. Couche de métadonnées

Les métadonnées sont volontairement séparées des fichiers.

Deux bases sont utilisées :

- **MongoDB** conserve les métadonnées techniques liées aux pipelines, aux jeux de données et aux traitements réalisés,
- **PostgreSQL** centralise les informations relationnelles sur les fichiers audio et leurs annotations.

Cette séparation facilite les recherches, le suivi des traitements et les futures évolutions de la plateforme.

4.3. Couche de traitement

Les traitements de données sont organisés sous forme de pipelines ETL indépendantes :

- téléchargement des jeux de données,
- ingestion dans le Data Lake,
- prétraitement audio et construction des échantillons d'apprentissage,
- construction des jeux d'entraînement.

Cette organisation permet d'exécuter chaque étape indépendamment et facilite leur maintenance.

L'ensemble des pipelines est actuellement développé en Python. L'architecture prévoit néanmoins leur migration vers **Apache Spark** afin de prendre en charge des volumes de données plus importants sans remettre en cause l'organisation générale de la plateforme.

4.4. Couche Data Science

Les notebooks Jupyter permettent :

- d'explorer les données,
- de construire différents jeux d'entraînement,
- d'entraîner plusieurs modèles,
- de comparer leurs performances.

Toutes les expérimentations sont suivies automatiquement par **MLflow**, qui enregistre paramètres, métriques et artefacts des modèles afin de garantir leur reproductibilité.

Un service **PostgreSQL** dédié constitue le backend de **MLflow** et les artefacts sont stockés dans **MinIO**.

4.5. Couche applicative

Le meilleur modèle sélectionné est exporté manuellement depuis MLflow au format **TensorFlow .keras**, puis intégré à une API **FastAPI**.

Une interface **Streamlit** permet aux utilisateurs de déposer un fichier audio, de récupérer les fichiers générés (MIDI et partition PDF) et de visualiser des représentations graphiques (piano-roll SVG et partition SVG).

Le déploiement de cette couche applicative est entièrement automatisé par une pipeline **GitHub Actions**, qui exécute :

- les tests automatisés,
- les contrôles qualité (**Ruff** et **MyPy**),
- la vérification de la couverture de tests,
- la construction de l'image **Docker**,
- sa publication sur **GitHub Container Registry**,
- le déploiement automatique sur **Hugging Face Spaces**.

Cette séparation entre plateforme Data et couche applicative permet de faire évoluer indépendamment les pipelines de préparation des données, les expérimentations de Machine Learning et l'application destinée aux utilisateurs.

5. Mise en oeuvre du Data Lake et des processus ETL

5.1. Collecte et intégration des données

Les données utilisées pour l'entraînement des modèles proviennent de deux corpus publics. Afin de garantir la reproductibilité des traitements, les données ne sont jamais manipulées directement.

La collecte est réalisée en deux étapes :

1. Pipeline de téléchargement

Les jeux de données sont téléchargés puis décompressés localement.

2. Pipeline d'ingestion

Cette seconde étape alimente l'infrastructure de données en répartissant les informations selon leur nature :

- les fichiers audio et annotations originaux? sont déposés dans le bucket **raw** du Data Lake **MinIO**,
- les métadonnées descriptives des fichiers sont enregistrées dans **PostgreSQL**,
- les annotations musicales sont extraites dans **MongoDB** afin de faciliter leur exploitation par les traitements ultérieurs.

Cette séparation entre données binaires et métadonnées simplifie les recherches et la traçabilité.

L'ensemble des données utilisées est constitué de jeux de données publics, aucune donnée personnelle n'est collectée ni traitée. Les exigences du RGPD sont donc respectées par conception.

5.2. Organisation du Data Lake

Le stockage repose sur **MinIO**, utilisé comme implémentation d'un Data Lake compatible avec l'API S3.

Les données sont réparties dans plusieurs buckets spécialisés.

Bucket	Contenu
raw	fichiers audio et annotations d'origine
processed	audios nettoyés, caractéristiques extraites et échantillons d'apprentissage
mlflow	artefacts produits lors des expérimentations
output	réservé aux futurs résultats produits par l'API

Cette organisation présente plusieurs avantages :

- conservation permanente des données sources,
- séparation claire entre données brutes et données transformées,
- possibilité de reconstruire intégralement les jeux d'entraînement,
- simplification de la maintenance et des sauvegardes.

Le Data Lake constitue ainsi le référentiel unique de stockage des objets volumineux manipulés par la plateforme.

5.3. Construction des données d'entraînement

Après leur ingestion, les fichiers audio traversent une chaîne de prétraitement entièrement automatisée.

La pipeline de prétraitement réalise successivement :

- le nettoyage du signal audio,
- la normalisation des amplitudes,
- l'extraction des représentations fréquentielles (CQT notamment),
- l'alignement temporel avec les annotations musicales,
- la génération d'échantillons destinés au Machine Learning.

Les nouveaux objets produits sont enregistrés dans le bucket **processed**.

Parallèlement, MongoDB conserve les métadonnées permettant de reconstituer précisément les traitements réalisés :

- paramètres de la pipeline,
- audios traités,
- échantillons générés.

Cette approche garantit la reproductibilité complète des jeux de données d'entraînement.

5.4. Gestion des métadonnées

L'architecture distingue volontairement les données métier des métadonnées techniques.

5.4.1. PostgreSQL

PostgreSQL centralise les informations relationnelles sur les ressources manipulées par la plateforme :

- références des fichiers audio,
- références des annotations,
- informations descriptives associées.

Cette base facilite les recherches structurées et garantit la cohérence des données.

5.4.2. MongoDB

MongoDB conserve les informations évolutives produites par les différentes pipelines :

- historique des traitements,
- paramètres d'exécution,
- composition des datasets,
- relations entre fichiers, échantillons et pipelines.

Ce choix est adapté à des documents dont la structure évolue au fil du projet.

La combinaison de **PostgreSQL** et **MongoDB** permet ainsi d'associer la robustesse d'un modèle relationnel à la flexibilité d'une base documentaire.

6. Préparation à la montée en charge

Le volume actuel des données permet l'exécution des pipelines sur une seule machine en Python.

Toutefois, l'architecture a été pensée dès sa conception pour permettre une évolution vers des traitements distribués.

Une infrastructure Apache Spark est déjà intégrée au projet via Docker Compose et comprend un noeud Spark Master et deux Spark Workers.

Cette infrastructure n'est pas encore utilisée en production. Elle constitue néanmoins le socle technique de la prochaine évolution du projet.

L'objectif est de remplacer progressivement les pipelines Python de prétraitement par des traitements distribués exécutés par Spark afin de :

- paralléliser le nettoyage des fichiers audio,
- accélérer l'extraction des caractéristiques,
- réduire les temps de traitement lors de l'augmentation du volume de données.

Le choix d'intégrer dès maintenant cette infrastructure permet d'anticiper les besoins futurs sans remettre en cause l'architecture existante.

7. Infrastructure as Code et reproductibilité

L'ensemble de l'environnement de développement est défini sous forme de code à l'aide de **Docker Compose**. Cette approche permet de reconstruire automatiquement une plateforme identique sur toute machine disposant de **Docker**.

L'infrastructure comprend l'ensemble des composants nécessaires au cycle de vie du projet :

- **MinIO** servant de Data Lake,
- **MongoDB** pour le stockage documentaire des métadonnées,
- **PostgreSQL** pour les données relationnelles,
- **PostgreSQL** dédié au suivi des expérimentations MLflow,
- **MLflow** pour le suivi des expériences de Machine Learning,
- **Apache Spark** (Master et Workers) destiné aux futurs traitements distribués,
- **FastAPI** et **Streamlit** constituant la couche applicative,
- **Mongo Express** et **pgAdmin** facilitant l'administration des bases de données.

La configuration des services est centralisée dans un fichier **.env**, permettant de séparer les paramètres d'exécution (identifiants, ports, buckets, URI) du code source. Un fichier **.env.example** est fourni afin de documenter l'ensemble des variables nécessaires à la reconstruction de l'environnement.

Les dépendances Python sont gérées avec **uv**, garantissant une installation reproductible grâce au verrouillage des versions (**uv.lock**).

Cette approche présente plusieurs avantages :

- environnement identique pour tous les développeurs,
- installation rapide de la plateforme,
- réduction des erreurs liées aux différences de configuration,

- simplification de la maintenance.

Pour reconstruire l'intégralité de l'environnement, exécuter les commandes :

```
# Création du fichier .env
cp .env.example .env

# Construction de l'infrastructure
docker compose up -d
```

L'ensemble des services est alors automatiquement déployé et interconnecté.

8. Industrialisation et qualité logicielle

La couche applicative fait l'objet d'une industrialisation complète grâce à une chaîne d'intégration et de déploiement continu implémentée avec **GitHub Actions**.

La pipeline CI/CD s'exécute à chaque push sur la branche **main** ou lors d'une Pull Request c'est-à-dire ?.

8.1. Validation du code

La pipeline réalise successivement :

- installation reproductible de l'environnement Python avec **uv**,
- exécution des tests unitaires et d'intégration,
- calcul de la couverture de tests avec seuil minimal imposé,
- analyse statique du code avec **Ruff**,
- vérification du typage avec **MyPy**.

Cette étape garantit que chaque évolution respecte les exigences de qualité définies pour le projet.

8.2. Construction et publication de l'application

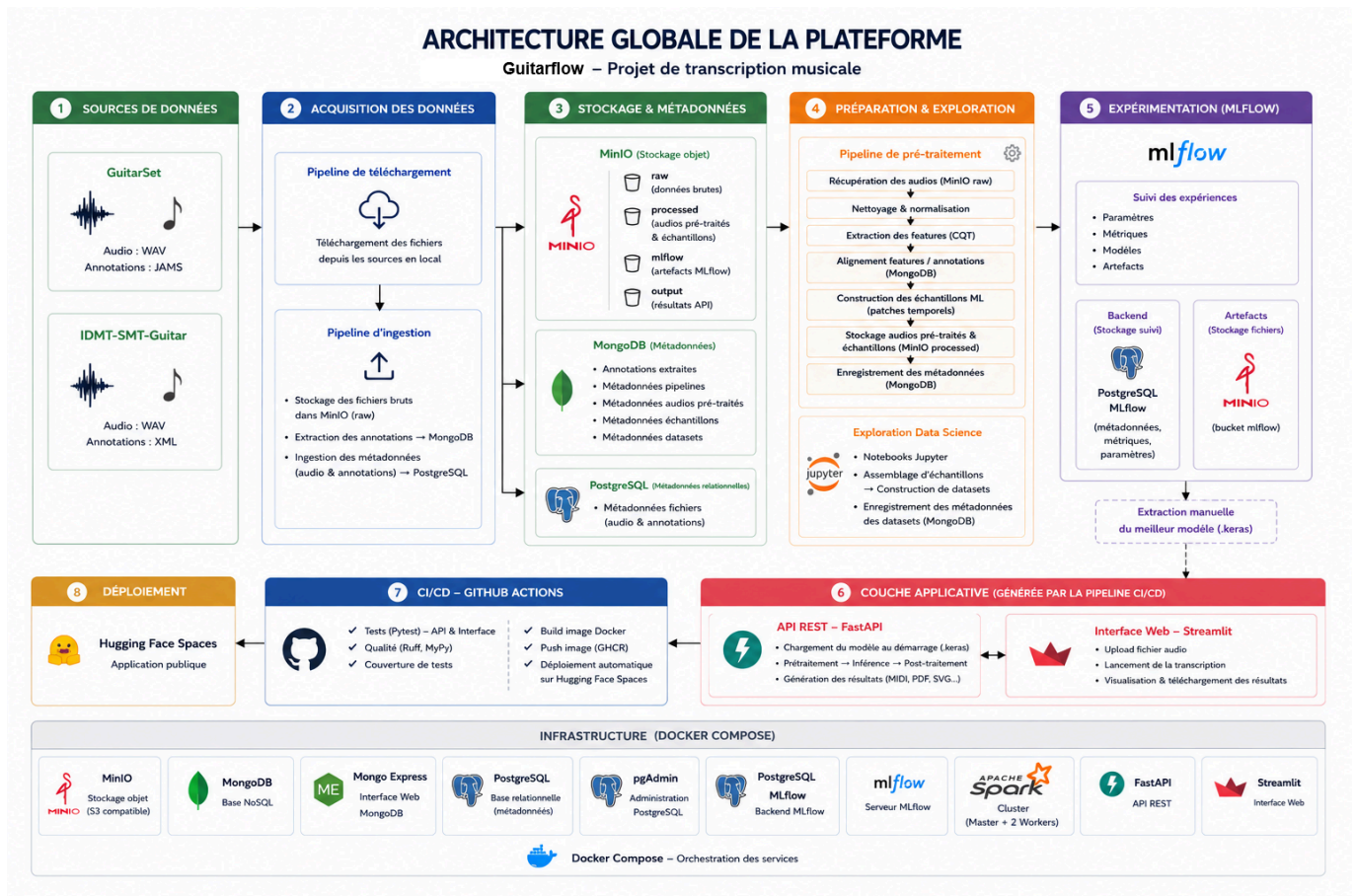
Lorsque les validations sont satisfaites :

- une image Docker est construite automatiquement,
- l'image est publiée sur **GitHub Container Registry (GHCR)**,
- le dépôt **Hugging Face Spaces** est mis à jour automatiquement afin de redéployer l'application.

Le déploiement de l'application ne nécessite ainsi aucune intervention manuelle.

Cette automatisation garantit la cohérence entre le code source, l'image distribuée et l'application accessible aux utilisateurs.

9. Schéma synthétique



10. Évolutivité de la plateforme

L'architecture a été conçue pour permettre des évolutions sans remise en cause de son organisation générale.

Plusieurs axes d'amélioration sont déjà identifiés.

10.1. Calcul distribué

Les pipelines ETL sont actuellement exécutées en Python.

L'infrastructure Spark déjà présente permettra, dans une version ultérieure, de distribuer les traitements de prétraitement audio afin d'améliorer les performances sur des volumes de données plus importants.

10.2. MLOps

Les expérimentations sont déjà suivies avec MLflow.

L'évolution naturelle de la plateforme consiste à automatiser davantage le cycle de vie des modèles, notamment :

- sélection automatique du meilleur modèle,
- validation avant mise en production,
- déploiement automatisé vers l'API.

10.3. Valorisation des résultats

Le bucket **output**, déjà prévu dans le Data Lake, pourra être utilisé pour archiver les productions générées par l'API (MIDI, partitions, représentations graphiques), facilitant ainsi leur partage ou leur réutilisation.

Ces évolutions pourront être réalisées sans modifier les fondements de l'architecture actuelle.

11. Conclusion

L'architecture conçue répond aux exigences fonctionnelles et techniques du projet tout en préparant ses évolutions futures.

Elle repose sur une séparation claire entre les différentes responsabilités :

- stockage des données dans un Data Lake,
- gestion des métadonnées,
- pipelines ETL de préparation des données,
- tracking des expérimentations des modèles,
- déploiement automatique d'une application web de transcription musicale.

Cette organisation garantit la reproductibilité des traitements, la traçabilité des données, la maintenabilité du projet et son évolutivité.

L'ensemble de cette infrastructure est intégralement décrit sous forme de code, reproductible via **Docker Compose**.